Hands-On Workshop: CI and CD for the Cars CRUD Application

Goal: Use GitHub Actions to test the FastAPI Cars application automatically, then build and publish its Docker image to Docker Hub when the tests pass.

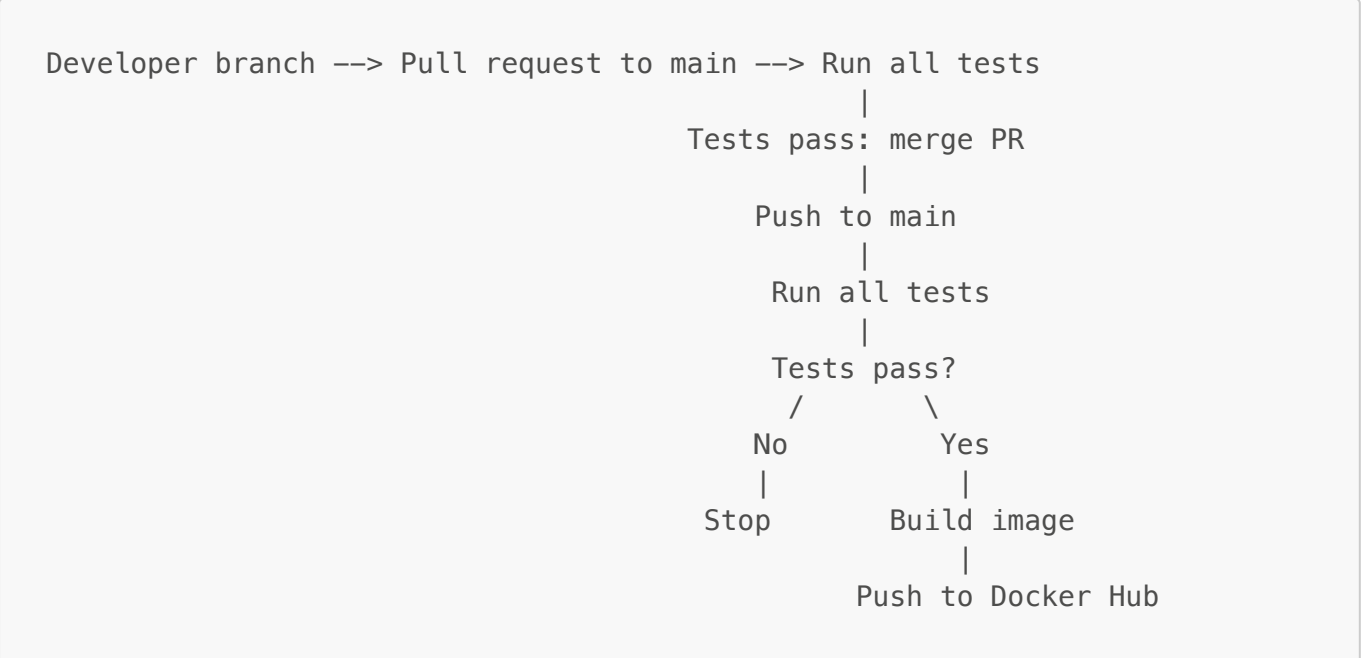
This is a **follow-along exercise**. The workflow below is complete: copy it, run it, inspect the results, and try the failure exercise.

1. What we are building

Our workflow has two responsibilities:

- **Continuous integration (CI):** Run unit, integration, API, and end-to-end (E2E) tests whenever a pull request targets `main`, and again whenever a commit is pushed to `main`.
- **Continuous delivery (CD):** After the tests pass on `main`, build the Cars CRUD Docker image and publish it to **Docker Hub**.

For this workshop, **publishing an image is our delivery step**. We are **not deploying or starting the application on a production server**.



Event	Unit + integration + API + E2E tests	Build and publish Docker image
Pull request targeting `main`	Yes	No (publishing job skipped)
Push or merge into `main`	Yes	Yes, **only if all tests pass**
Failed tests on `main`	Fail	No

2. Check your project

You should already have the Cars CRUD project on GitHub, with these files:

```
cars-crud/
├── app/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── api/
│   └── e2e/
├── Dockerfile
├── .dockerignore
├── requirements.txt
├── requirements-dev.txt
└── .github/
    └── workflows/
        └── tests.yml             # From the previous GitHub Actions
exercise
```

Prerequisites: You have completed the earlier software testing, Docker, and GitHub Actions exercises. You need a GitHub account, a Docker Hub account, and a Dockerfile that builds the application. If you used the earlier MySQL/Compose exercise, keep `compose.yaml`; it is **not required** for this workflow.

Confirm that your tests work locally

From the project root, inside your Python virtual environment:

```
python -m pip install -r requirements-dev.txt
python -m playwright install chromium
python -m pytest tests/unit tests/integration tests/api -q
python -m pytest tests/e2e -q
```

The existing tests use **temporary SQLite databases**, even if you have also configured MySQL for Docker Compose. The E2E test fixture automatically starts and stops a local FastAPI server, so you do **not** need to run Uvicorn yourself during the GitHub workflow.

Create a working branch

```
git switch main
git pull --ff-only origin main
git switch -c cars-ci-cd
```

If you already have a branch named `cars-ci-cd`, switch to it or choose another branch name.

3. Prepare Docker Hub

1. Sign in at https://hub.docker.com/.
2. Create a repository named `cars-crud` under your own Docker Hub username. A public repository is simplest for this exercise.
3. Open your Docker account settings, find **Personal access tokens**, and generate a token with **Repo Read & Write** permissions. Give it a recognizable description such as `cars-github-actions`.
4. Copy the token when it appears. **Do not commit the token, write it in a YAML file, or share it in screenshots.**

Your published image will have a name similar to:

```
YOUR_DOCKER_ID/cars-crud:latest
```

The part before `/cars-crud` is your **Docker Hub username**, not necessarily your GitHub username.

4. Save the Docker Hub credentials as GitHub Secrets

In the **GitHub repository** (not Docker Hub):

1. Open **Settings > Secrets and variables > Actions**.
2. Select **New repository secret**.
3. Create these two secrets, using the **exact** names below:

GitHub secret	Value
`DOCKERHUB_USERNAME`	Your Docker Hub username
`DOCKERHUB_TOKEN`	The Docker Hub personal access token

GitHub Actions reads secrets with this syntax:

```
username: ${{ secrets.DOCKERHUB_USERNAME }}
password: ${{ secrets.DOCKERHUB_TOKEN }}
```

Important: The token belongs in GitHub Secrets, **never** directly in a committed workflow file. Although the username is not sensitive, we store both values as secrets here to keep the exercise consistent.

Checkpoint: You should see both **secret names** in the GitHub settings page. Their values will not be displayed.

5. Write the GitHub Actions workflow

Open the **existing** `.github/workflows/tests.yml`. Replace its contents with the following workflow. Do not create a second testing workflow unless you want the same tests to run twice.

```
name: Cars CRUD - Test and Publish

on:
```

```yaml
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  tests:
    name: Run application tests
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v7

      - name: Set up Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"

      - name: Install application and test dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements-dev.txt

      - name: Install Chromium for E2E tests
        run: python -m playwright install --with-deps chromium

      - name: Run unit tests
        run: python -m pytest tests/unit -q

      - name: Run integration tests
        run: python -m pytest tests/integration -q

      - name: Run API tests
        run: python -m pytest tests/api -q

      - name: Run E2E tests
        run: python -m pytest tests/e2e -q

  publish:
    name: Build and publish Docker image
    needs: tests
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v7

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v4
```

```
      - name: Log in to Docker Hub
        uses: docker/login-action@v4
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v7
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/cars-crud:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/cars-crud:sha-${{ github.sha
}}
```

Understand the important lines

Workflow instruction	Meaning
`on: pull_request`	Run the tests when a PR targets `main` or receives new commits.
`on: push`	Also run when changes arrive on `main`, including after merging a PR.
`branches: [main]`	Limit those events to the named target/updated branch.
`permissions: contents: read`	Give the workflow's GitHub token only repository read access.
`jobs.tests`	A job that runs the four test groups.
`runs-on: ubuntu-latest`	Use a GitHub-hosted Ubuntu machine.
`actions/checkout`	Download the source code onto that machine.
`actions/setup-python`	Install/select Python 3.12.
`requirements-dev.txt`	Install FastAPI and our Python testing dependencies.
`playwright install --with-deps chromium`	Install the browser and Linux libraries needed by E2E tests.
`needs: tests`	Wait for the test job and run only if it succeeds.
`if: ... push ... main`	Never publish from a pull request; publish only after a push to `main`.
`docker/setup-buildx-action`	Prepare Docker's image builder.
`docker/login-action`	Authenticate to Docker Hub using stored credentials.

Workflow instruction	Meaning
`docker/build-push-action`	Build `Dockerfile` and upload the image because `push: true`.
`tags:`	Publish `latest` and a tag linked to the Git commit SHA.

The second tag, `sha-<commit SHA>`, lets you identify exactly which revision produced an image; `latest` points to the most recently published successful build.

What about Docker Compose and MySQL?

Our **tests run against isolated SQLite files**, so this workshop does not need a MySQL container in GitHub Actions. The publishing job builds **the FastAPI application image**, not a Docker Compose bundle. If you later want to run it with MySQL, your Compose configuration must also provide the MySQL service and its storage.

What about the Azure Docker image proxy?

For students in China, pulling base images locally may be faster through the course's Azure proxy:

```
# Standard base image
# python:3.12-slim

# Optional local build through the course proxy
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/cache/library/python:3.12-slim -t cars-
crud:local .
```

This uses the `ARG BASE_IMAGE` setting in the Dockerfile from our Docker guide. **On GitHub-hosted runners, the normal Docker Hub base image (`python:3.12-slim`) is sufficient in typical circumstances**, so the workflow does not need a proxy build argument. Regardless of where the base image comes from, **the finished application image is pushed to your own Docker Hub account**.

6. Push the workflow and open a pull request

Commit the workflow from your `cars-ci-cd` branch:

```
git add .github/workflows/tests.yml
git commit -m "Test and publish Cars CRUD with GitHub Actions"
git push -u origin cars-ci-cd
```

If you modified your Dockerfile or other project files, commit those as well.

In GitHub:

1. Open **Pull requests > New pull request**.

2. Set **base:** `main` and **compare:** `cars-ci-cd`.
3. Create the PR and open **Checks** (or open the run from the **Actions** tab).
4. Open the **Run application tests** job and inspect the four testing steps.
5. Confirm the **Build and publish Docker image** job says **Skipped**. This is correct: a PR should not publish an image.

Checkpoint: The PR shows passing unit, integration, API, and E2E tests. Docker Hub has **not** received a new image from this PR run.

7. Prove that failing tests stop the workflow

Before merging, deliberately break one of your existing tests:

1. Open `tests/unit/test_schemas.py` and find `test_color_is_optional`.
2. Temporarily change:

   ```
   assert car.color == ""
   ```

   to:

   ```
   assert car.color == "Blue"
   ```

3. Commit and push the change on your PR branch:

   ```
   git add tests/unit/test_schemas.py
   git commit -m "Demonstrate a failing unit test"
   git push
   ```

4. Open the newest run in GitHub Actions. The **unit test step should fail**. Later test steps may be skipped because execution stopped at that failure. The publishing job remains skipped.
5. Restore `assert car.color == ""`, then commit and push the fix:

   ```
   git add tests/unit/test_schemas.py
   git commit -m "Restore passing unit test"
   git push
   ```

6. Confirm the **newest** PR run is green before merging.

Why this matters: A failed check signals that a code change is not ready to be merged. Do not merge the deliberately broken test.

8. Merge into `main` and watch the image publish

1. Merge the pull request into `main`.
2. Open the **Actions** tab and find the **new run triggered by `push` to `main`**. This is a **different run** from the PR check.
3. Watch **Run application tests** complete successfully.
4. Watch **Build and publish Docker image** begin **after** the tests succeed.
5. Open the publishing job logs and verify that **Log in to Docker Hub** and **Build and push Docker image** finished successfully.
6. Open your Docker Hub repository and verify two tags are present:

```
YOUR_DOCKER_ID/cars-crud:latest
YOUR_DOCKER_ID/cars-crud:sha-<full Git commit SHA>
```

Checkpoint: A successful push to `main` produces a new Docker Hub image. A failed test stops the publishing job instead.

9. Download and run the published image

On a computer with Docker installed, replace `YOUR_DOCKER_ID` with your username:

```
docker pull YOUR_DOCKER_ID/cars-crud:latest

docker run -d \
  --name cars-from-hub \
  -p 127.0.0.1:8000:8000 \
  --mount type=volume,source=cars-published-data,target=/data \
  YOUR_DOCKER_ID/cars-crud:latest
```

Open **http://localhost:8000/** or **http://localhost:8000/docs**, then create a car.

- The image includes FastAPI and the raw HTML/CSS/JavaScript frontend.
- The named volume persists the default **SQLite database** stored at `/data/cars.db`.
- This is a local demonstration using an artifact delivered through Docker Hub, **not** a production deployment.

If port 8000 is already in use, stop the old container first or map `127.0.0.1:8001:8000` and open http://localhost:8001/. For cleanup, use `docker stop cars-from-hub` followed by `docker rm cars-from-hub` (the named volume remains until removed separately).

10. Troubleshooting

Symptom	What to check
No workflow run appears on the PR	Ensure the file is `.github/workflows/tests.yml` and the PR targets `main`.
`No module named pytest`	Check the `requirements-dev.txt` installation step.
Chromium is missing or cannot launch	Run `python -m playwright install --with-deps chromium` on the Ubuntu runner.
The E2E server cannot start	Check E2E fixture errors; it starts FastAPI automatically using a temporary port.
Publishing says **Skipped** on a PR	Expected. Publishing is restricted to **push to `main`**.
Publishing says **Skipped** on `main`	Check whether the tests failed and inspect the `if` condition.
Docker Hub login returns `unauthorized`	Recheck `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`, token validity, and its permissions.
Image push returns `denied`	Verify that `cars-crud` exists in the correct Docker Hub namespace and the token has **Repo Read & Write** permission.
Docker build reports `Dockerfile` not found	Make sure `Dockerfile` is at the repository root and committed.
A published image is old	Check the latest workflow run and compare the `sha-...` image tag with the commit on `main`.
Docker Hub is slow on your local network	Use the Azure proxy for supported **base-image pulls**, as described above. It is not the destination for publishing your private application image.

11. Completion checklist

- ☐ My app has passing unit, integration, API, and E2E tests.
- ☐ My Docker Hub account has a `cars-crud` repository.
- ☐ The GitHub repository contains `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` secrets.
- ☐ `.github/workflows/tests.yml` runs tests for PRs targeting `main`.
- ☐ The same workflow runs tests again on pushes to `main`.
- ☐ The `publish` job depends on a successful `tests` job.
- ☐ The `publish` job runs only on a push to `main`.
- ☐ A deliberately failing test makes the PR check red; restoring it makes the check green.
- ☐ After merging, Docker Hub contains the new `latest` and commit-specific image tags.
- ☐ I can pull and run the published image locally.

Submission: Provide the GitHub pull-request link, a link to the successful GitHub Actions run on `main`, and the Docker Hub repository link. **Never submit your access token.**

Official references

- GitHub Actions workflow events
- GitHub Actions repository secrets
- Docker Hub access tokens
- Docker build and push action
- Playwright Python on GitHub Actions